



“Escaping Java's Allocation Gravity with Value Classes”

Clément de Tastes

Rémi Forax

Project Valhalla

- OpenJDK project
 - Language (Amber, Babylon)
 - Platform (Loom, Panama, Leyden, Lilliput, **Valhalla**)
- Started in 2014 (!!) by Brian Goetz & John Rose

“Java’s epic refactor”

Brian Goetz



“Codes like a class, works like an int”

Why Valhalla ?

- **Zero-cost abstraction**
- **Flat memory representation**
- **Primitives vs Objects**

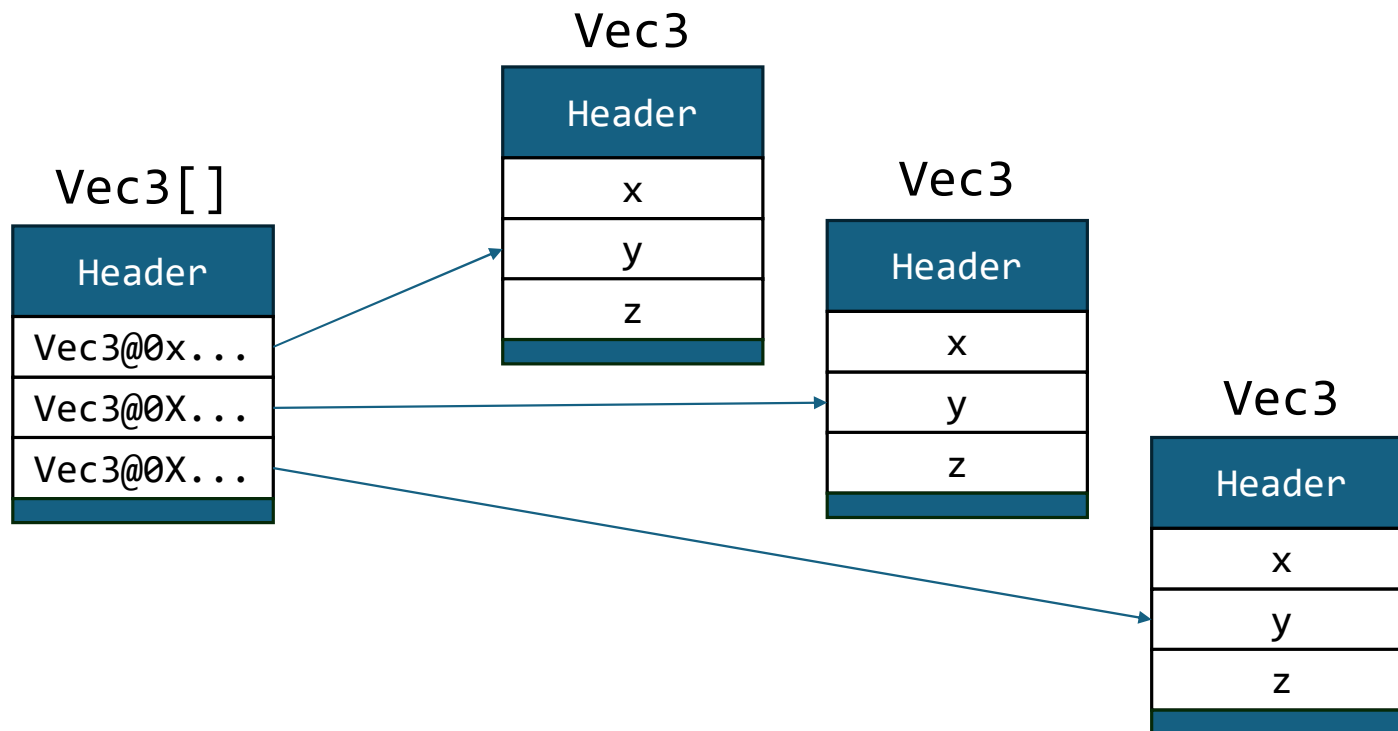
Zero-Cost Abstraction

We should not choose between abstraction and performance

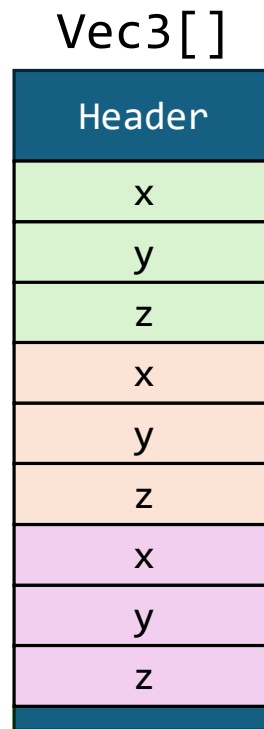
```
public class Mass {  
    private final double mass;  
    public Mass(double mass) {  
        if (mass < 0) {  
            throw new IllegalArgumentException("Mass must be strictly positive");  
        }  
        this.mass = mass;  
    }  
}
```

Memory Representation

```
record Vec3(double x, double y, double z) {}
```



Flat Memory Representation



Primitives vs Objects

Primitives are second class citizens

- **No abstraction**
 - no common array
 - cannot be used as generics
- **No method** (`toString()`, ...)
- `==` vs `equals()`

JEP 401 introduces **Value Objects** in the language

JDK 28

This release will be the Reference Implementation of version 28 of the Java SE Platform, as specified by [JSR 403](#) in the Java Community Process.

Status

The [main line](#) is open for bug fixes, small enhancements, and JEPs as proposed and tracked via the [JEP Process](#).

Early-access binaries, under the GPL, are available [here](#).

Features

JEPs targeted to JDK 28, so far

- 401: [Value Objects \(Preview\)](#)
- 535: [Shenandoah GC: Generational Mode by Default](#)
- 539: [Strict Field Initialization in the JVM \(Preview\)](#)
- 540: [Simple JSON API \(Incubator\)](#)
- 541: [Deprecate the macOS/x64 Port for Removal](#)
- 542: [PEM Encodings of Cryptographic Objects](#)

Available as a **preview** feature in JDK 28 early access build

8389219: Implement JEP 401: Value Objects (Preview) #31120

Code ▾

Closed MrSimms wants to merge 2934 commits into `openjdk:master` from `MrSimms:8317277`

Conversation 156 Commits 2934 Checks 22 Files changed 1888 +208 011 -13 161



MrSimms commented on May 11 • edited by openjdk Bot Member ...

This pull request implements the first [preview](#) of [JEP 401: Value Objects \(Preview\)](#). This pull request also includes the implementation of [JEP 539: Strict Field Initialization in the JVM \(Preview\)](#). That work was implemented in the same code base because JEP 401 depends on strict field initialization.

Sub-reviews, for spreading review debates are here:

- [JDK-8317277](#): Java language implementation of value objects
 - [8317277: Java language implementation of value objects](#) #31121
- [JDK-8317278](#): JVM implementation of value objects
 - [8317278: JVM implementation of value objects](#) #31122
- [JDK-8317279](#): Standard library implementation of value objects
 - [8317279: Standard library implementation of value objects](#) #31123

This is the "*master pull request*" for the initial preview of [JEP 401](#). Comments and review for a change this large will not scale well in a single pull request. This pull request serves as the vehicle for sign-off and integration into `jdk/master`. Review comments should be directed to the appropriate "*sub-review pull request*" listed above.

Note

The "*sub-review pull requests*" contain the same full set of code changes as this "*master pull request*" to preserve the full implementation context; the language compiler, JVM, and standard library changes are intertwined. The separate pull requests exist only to subdivide the review and related discussion by area.

Any resulting code changes should be made in `valhalla/lworld`.

Reviewers

17 more reviewers

- borodulinartm
- dfuch
- mgronlun
- lahodaj
- plummercj
- dean-long
- jddarcy
- iklam
- jaikiran
- liach
- DanHeidinga
- sspitsyn
- mcimadamore
- coleenp
- dcubed-ojdk
- xmas92
- viktorklang-ora

Clément de Tastes

- Tech Lead @ SCIAM



- @cdetastes.bsky.social
- github.com/CodeSimcoe
- blog.sciam.fr
- www.toulon.dev
- Java ☕ — JavaFX — Quarkus
- I brew my own beer 🍺





Rémi Forax

- Associate Professor at University Gustave Eiffel (Paris)
- github.com/forax
- Java ☕ specs for invokedynamic, lambda, module, record, pattern-matching, value-type
- I maintain several open-source projects nobody use

Disclaimer

These are **preview** or experimental features



What is a Value Class ?

Instances / references / objects are **values**, not **pointers**

New keyword : **value**

```
value record Vec3(double x, double y, double z) {}
```

A value class instance is passed “by value”

```
static Point translate(Point p, double dx, double dy) {  
    return new Point(p.x + dx, p.y + dy);  
}
```

At runtime, in “pseudo Java”

```
static (double, double) translate((double x, double y), double dx, double dy) {  
    return (x + dx, y + dy);  
}
```

May eliminate allocation (scalarization)

Value Objects are :

Identity-free (no address in memory, no header)

- **Unmodifiable** (fields are `final`, class is `final`)
- Stored and passed **by value** rather than by pointer

Goals :

- **No pointer indirection** for small data structures
- **No overhead** for heap allocation

Demo 01 & 02

Value Classes require strict initialization

- a **value object** must never have an observable state that changes
- there's a place we can mutate **final** fields : the constructor
- JEP 513 : Flexible Constructor Bodies (*Java 25*)
- all fields can - *and must* - be initialized before the call to **super()**

 Demo 03

Are **Value Class** instances objects ?

- Yes !
- They can even implement interfaces...
- ... or extend an abstract class

 **Demo 04**

Storing **Value Class** instances in fields/array

It works, but the JVM might not be able to optimize

```
class Ray {  
    Vec3 origin;  
    // ...  
}
```

```
class Ray {  
    double origin_x;  
    double origin_y;  
    double origin_z;  
    // ...  
}
```

- Reading/storing the value instance **origin** requires **several reads/writes**
- The VM spec mandates reference reads/writes to be **atomic**
- **64** bit CPU vs **192** bits here

Storing Value Class instances in fields/array

`null` marker required : 7 bytes of useful payload (56 bits)

```
class Ray {  
    boolean origin_null;  
    double  origin_x;  
    double  origin_y;  
    double  origin_z;  
    // ...  
}
```

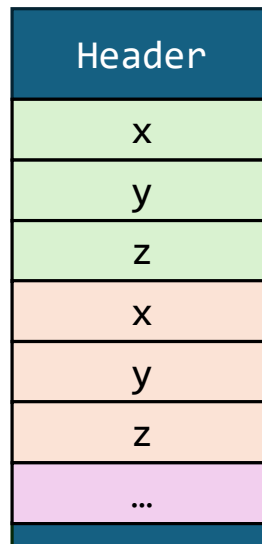
Flattening on heap : storing `Vec3[]`

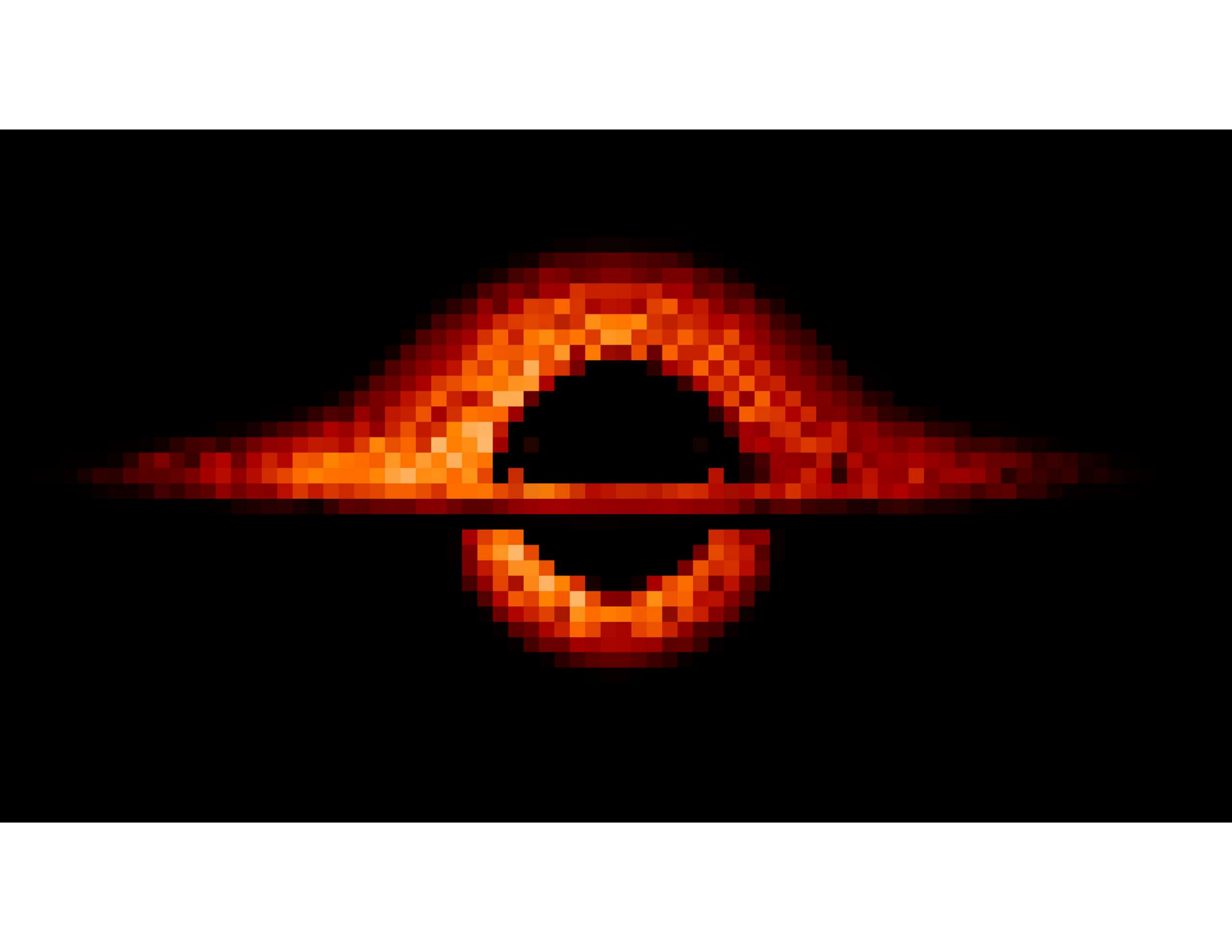
`Vec3` is too “large” to be flattened

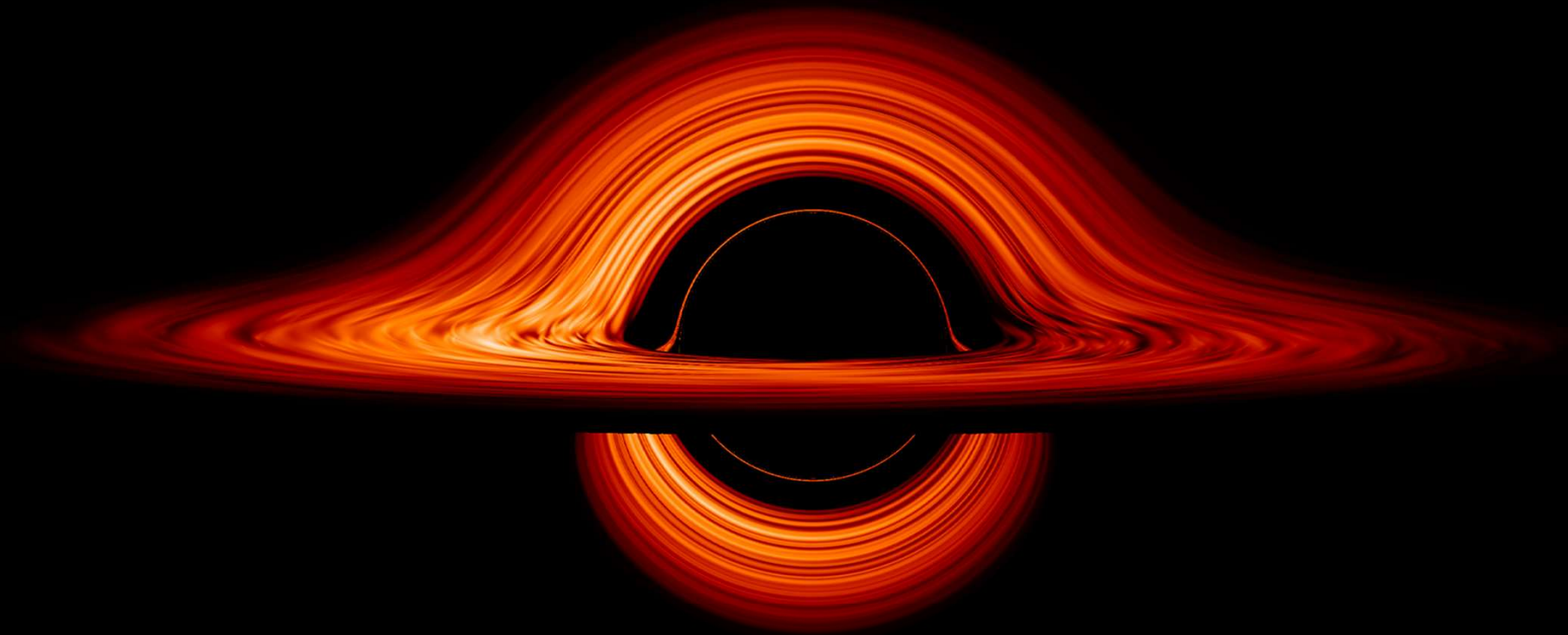
A cheap version can

```
record Vec3(byte x, byte y, byte z) {}
```

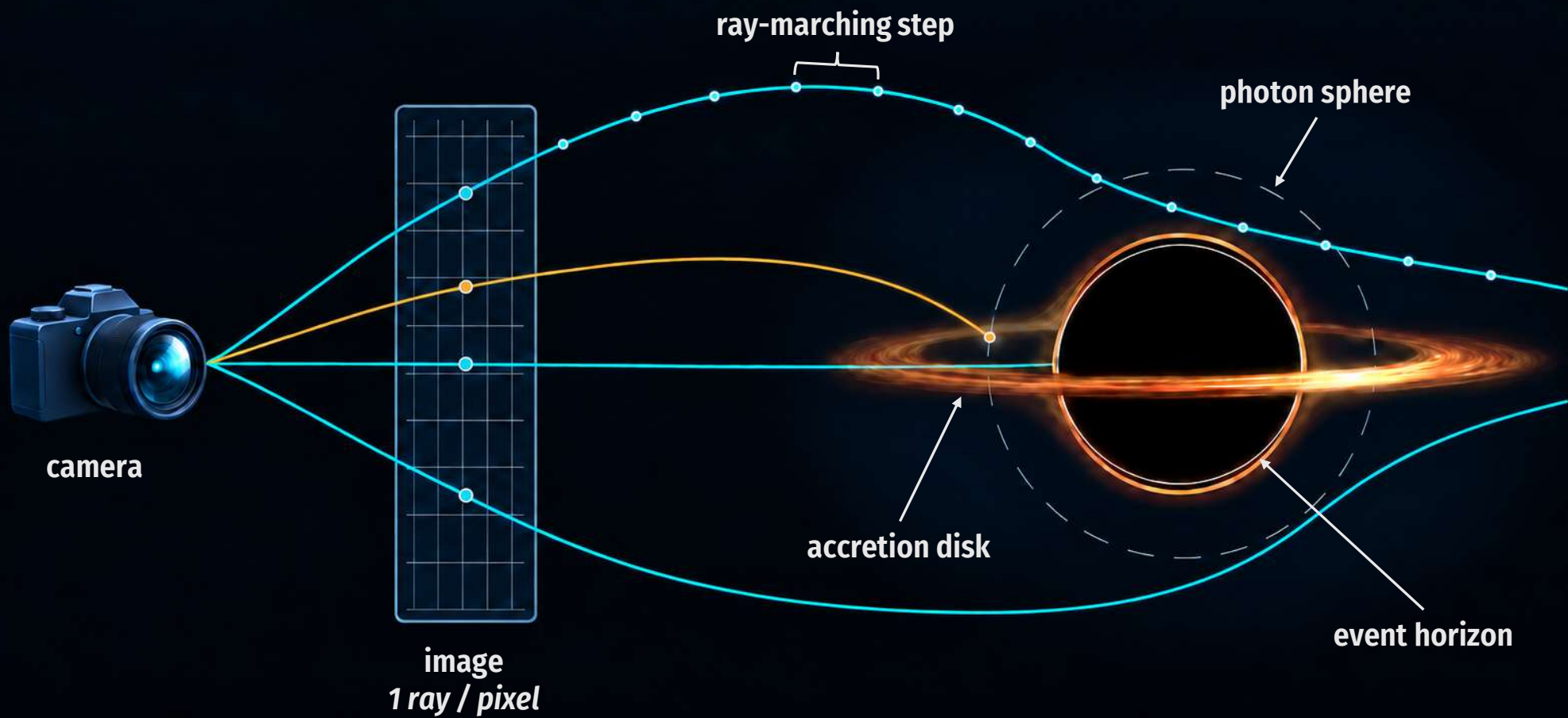
`Vec3[]`







Let's generate a blackhole, shall we ?



Defining the Domain Model

Trace thousands of **Ray** from the camera

```
record Ray(Vec3 origin, Vec3 direction) {}
```

Use **Vec3** to represent and update positions and directions

```
record Vec3(double x, double y, double z) {}
```

Accumulate encountered light and produce the pixel's final **Color**

```
record Color(double r, double g, double b) {}
```

Algorithm's snippet

```
var vel = forward
    .add(right.mul(u))
    .add(trueUp.mul(v))
    .norm();
```

```
double velX = forwardX + rightX * u + trueUpX * v;
double velY = forwardY + rightY * u + trueUpY * v;
double velZ = forwardZ + rightZ * u + trueUpZ * v;

double mag = Math.sqrt(
    velX * velX +
    velY * velY +
    velZ * velZ
);

if (mag != 0.0) {
    double invMag = 1.0 / mag;

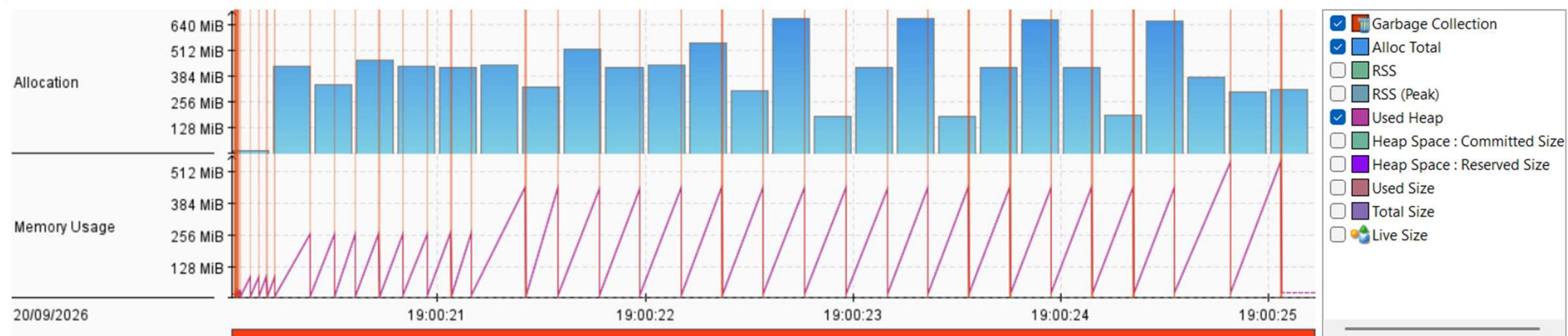
    velX *= invMag;
    velY *= invMag;
    velZ *= invMag;
}
```

Benchmark

Identity vs Value

Benchmark	Mode	Cnt	Score	Error	Units
renderWithIdentity	avgt	5	52,593 ±	1,389	ms/op
gc.alloc.rate	avgt	5	2022,373 ±	53,553	MB/sec
gc.alloc.rate.norm	avgt	5	111536309,830 ±	9,108	B/op
gc.count	avgt	5	128,000		counts
gc.time	avgt	5	51,000		ms
renderWithValue	avgt	5	48,566 ±	1,727	ms/op
gc.alloc.rate	avgt	5	4,151 ±	0,147	MB/sec
gc.alloc.rate.norm	avgt	5	211368,714 ±	7,780	B/op
gc.count	avgt	5	≈ 0		counts

Identity

[illegible]

Value

Class	Max Live Count	Max Live Size	Live Size Increase	Alloc Total	Total Allocation (%)
BlackholeGenerator\$Vec3				306 MiB	89,9 %
BlackholeGenerator\$Color				22,3 MiB	6,56 %
int[]				11,2 MiB	3,28 %
byte[]				221 KiB	0,0634 %
java.lang.reflect.Method				108 KiB	0,031 %
jdk.internal.classfile.impl.LabelImpl				56,8 KiB	0,0163 %
jdk.internal.classfile.impl.DirectMethodBuilder				56 KiB	0,0161 %
jdk.internal.classfile.impl.BufWriterImpl				52 KiB	0,0149 %
java.net.InetSocketAddress				48,2 KiB	0,0138 %
char[]				36,2 KiB	0,0104 %
java.lang.String				36,1 KiB	0,0104 %
java.lang.invoke.LambdaForm\$Name				36,1 KiB	0,0103 %
jdk.internal.classfile.impl.DirectFieldBuilder				36 KiB	0,0103 %



This is not “*just*” a performance improvement

Changes in the JDK

- `@ValueBased` -> Value Types
- `java.lang` : `Integer`, `Long`, `Float`, `Double` ...
- `java.util` : `Optional`, `OptionalInt`, `OptionalLong` ...
- `java.time` : `LocalDate`, `LocalTime`, `Duration`, `Instant` ...
- **JEP 390: Warnings for Value-Based Classes (JDK 16)**

 **Demo 05**

Java compiler and VM (*JEP 539*)

When compiling a value class, the compiler :

- removes the `ACC_IDENTITY` modifier bit of the class file
- adds `ACC_STRICT_INIT` on all fields
- no need to recompile old external code using value class...
- ... but this would help : `LoadableDescriptors` is added
- these classes are early loaded by VM

Conclusion

Value Types

- Encapsulation (POO) + performances des primitifs
- Pas juste une “amélioration” de performance
- Next steps :
 - explore more flattening opportunities
 - bridge the gap between primitives and value objects



<https://github.com/CodeSimcoe/blackhole-devoux>

Flattening types

	null marker	null free
atomic	56 bits	64 bits*
non-atomic	must be strict final*	no restriction*

(*) not yet fully implemented

Valhalla Roadmap



- JEP 401 : Value Classes & Objects
 - Null-Restricted and Nullable Types
 - Null-Restricted Value Class Types
- JEP 402 : Enhanced Primitive Boxing (`int` $\approx$ `Integer`!)
- Parametric JVM (`List<ValueType>`)

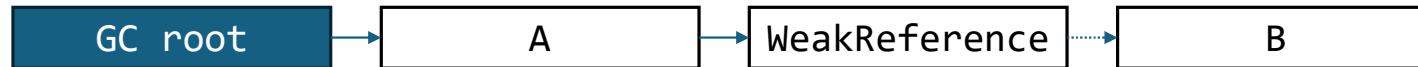
synchronized

- **synchronized** needs a bit in the header
- **A value class has no header**

hashCode()

- Default `hashCode()` picks a random number and stores it in the header
- A value class *still* has no header
- Relies on the fields' values to compute the hash

Weak References



- `WeakReference` leaves **B** collectible by GC (*if only reference*)
- **Can't make a reference to a non-existing address in memory**
- **Throws** `(No)IdentityException`